Optimizing Job Shop Scheduling via Graph Neural Networks and Group Relative Policy Optimization

Kerstin Sun
Department of Computer Science
Rice University
ks256@rice.edu

Heng Jui Hsu
Department of Computer Science
Rice University
hh83@rice.edu

Cheng Han Lin
Department of Computer Science
Rice University
cl278@rice.edu

1 Problem Definition

1.1 The Job Shop Scheduling Problem (JSSP)

We consider the standard Job Shop Scheduling Problem, defined by a set of jobs $\mathcal{J} = \{J_1, \ldots, J_n\}$ and a set of machines $\mathcal{M} = \{M_1, \ldots, M_m\}$. Each job J_i consists of a sequence of operations $O_{i,1} \to O_{i,2} \cdots \to O_{i,k}$ that must be processed in a strict order (conjunctive constraints). Furthermore, each operation O_{ij} requires a specific machine and occupies it for a fixed duration p_{ij}, during which no other operation can use that machine (disjunctive constraints).

The objective is to determine a schedule—a start time for every operation—that minimizes the **Makespan** ($C_{\max}$), defined as the completion time of the last operation across all jobs:

$$\text{Minimize } C_{\max} = \max_{i \in \mathcal{J}}(C_i) \tag{1}$$

Following the formulation in [1], we represent the state s_t as a **Disjunctive Graph** $G = (\mathcal{O}, \mathcal{C}, \mathcal{D})$. The scheduling process is treated as a sequential decision-making task where the agent iteratively selects an operation from the set of eligible nodes (those whose predecessors are completed) until all operations are scheduled.

2 Motivation

While JSSP is a well-defined combinatorial problem, solving it efficiently at scale remains an open challenge. Exact solvers (e.g., CP-SAT) are computationally prohibitive for large instances (NP-hard), and heuristic rules (e.g., FIFO, MWKR) often yield sub-optimal results due to their inability to look ahead.

Recent constructive Deep Reinforcement Learning (DRL) approaches [1] have shown promise by learning to dispatch operations end-to-end. However, the standard algorithm used—Proximal Policy Optimization (PPO) [3]—suffers from a structural flaw when applied to scheduling: **Reliance on a Critic.**

In standard PPO, a Value Network (Critic) must estimate the expected final makespan $V(s_t)$ from a partial schedule state s_t. In JSSP, the reward is sparse and delayed; a decision made at $t = 1$ might not reveal its true cost until $t = 100$. This makes training an accurate Critic notoriously difficult. When the Critic's estimate is noisy, the calculated Advantage (A_t) becomes unreliable, leading to high variance gradients and training instability.

Preprint. Under review.

Our project is motivated by a simple hypothesis: **If the Critic is the weak link, we should remove it.** By adopting Group Relative Policy Optimization (GRPO) [2, 5], we can eliminate the need for value function approximation entirely. Instead of comparing a schedule against a "guessed" baseline, we can compare it against a group of actual peer schedules, providing a robust, variance-reduced learning signal.

3 Proposed Methodology

Our proposed framework replaces the Actor-Critic architecture with a group-based policy optimization scheme. The method consists of a Graph Neural Network (GNN) for state encoding and the GRPO algorithm for policy updates.

3.1 Model Architecture: Critic-Free GNN

We adopt the Graph Isomorphism Network (GIN) [4] architecture from [1] to embed the disjunctive graph, but with a significant modification: we remove the Value Head.

- **Encoder:** A GIN extracts node embeddings h_v by aggregating features from both conjunctive (job) and disjunctive (machine) neighbors.
- **Policy Head:** A Multilayer Perceptron (MLP) maps the embeddings of eligible operations to a probability distribution $\pi_\theta(a|s)$.
- **No Value Net:** Unlike standard PPO, we do not initialize or train a second network to estimate $V(s)$.

3.2 GRPO Training Loop

To estimate the advantage of an action without a Critic, we utilize the "Group" property of GRPO. The training iteration proceeds as follows:

1. **Group Sampling:** For a given instance I, we sample G independent trajectories (complete schedules) $\{o_1, o_2, \ldots, o_G\}$ from the current policy $\pi_{\theta_{old}}$.
2. **Outcome Evaluation:** We compute the final makespan $C_{\max}(o_i)$ for each schedule. This serves as the ground-truth performance metric.
3. **Relative Advantage:** The advantage of the i-th schedule is computed by standardizing its makespan against the group mean:
$$\hat{A}_i = \frac{\text{Mean}(\{-C_{\max}\}) - (-C_{\max}(o_i))}{\text{Std}(\{-C_{\max}\}) + \epsilon} \tag{2}$$
(Note: We use negative makespan because lower is better).
4. **Objective Function:** The policy π_θ is updated to maximize the likelihood of schedules that performed better than the group average, constrained by a KL-divergence term:
$$\mathcal{L}_{GRPO} = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\left(\frac{\pi_\theta(o_i)}{\pi_{\theta_{old}}(o_i)}\hat{A}_i\right) - \beta D_{KL}(\pi_\theta||\pi_{ref})\right] \tag{3}$$

This methodology ensures that the agent learns strictly from verifiable outcomes (completed schedules) rather than unstable intermediate guesses.

4 Dataset

The dataset for this project consists of two parts: synthetic instances for training and classic benchmark sets for performance evaluation.

Following the convention in deep reinforcement learning for JSSP, we will use synthetic instances generated based on Taillard's distribution to train the GNN-based agent. These instances are generated covering problem scales such as 10×10, 15×15, and 20×20.

To evaluate generalization capability of the GRPO-trained policy, we will test the model on classic benchmarks without any additional fine-tuning. These include:

- **Small-to-medium scale:** FT06/10/20, LA01–LA40.
- **Large-scale and high-difficulty:** ORB01–10, SWV01–20, TA01–80.

5 Evaluation Metrics

To quantitatively assess the performance of the proposed GRPO-GNN framework, we will employ the following metrics:

- **Makespan ($C_{\max}$):** The total time required to complete all jobs. This is the primary objective of the JSSP.
- **Relative Scheduling Error (RSE) (%):** The relative gap between the model's makespan and the optimal solution ($C^*_{\max}$). It reaches 0% when an optimal solution is found.

$$\mathrm{RSE} = \left(\frac{C_{\max} - C^*_{\max}}{C^*_{\max}} \right) \times 100\,\%$$

- **Optimum Rate (OR):** OR represents the percentage of instances where the model reaches the optimal solution.

5.1 Baselines

Our primary baseline is **L2D (PPO)** [1], the original GNN-based dispatching agent trained with Proximal Policy Optimization. Since our core contribution is replacing PPO with GRPO on the same architecture, this comparison directly validates our hypothesis.

As additional references, we include:

- **OR-Tools** (Google): an exact constraint programming solver with a 3600-second time limit per instance, serving as a near-optimal upper bound.
- **Priority Dispatching Rules (PDRs)**: *Shortest Processing Time* (SPT), *Most Work Remaining* (MWKR), and *Most Operations Remaining* (MOR), representing heuristic lower bounds.

For the public benchmarks, we additionally compare against best-known solutions from the literature.

6 Project Timeline and Team Responsibilities

6.1 Team Roles

- **Heng Jui Hsu** – GNN Encoder & Environment: disjunctive graph representation, GIN encoder, JSSP environment setup.
- **Kerstin Sun** – GRPO Training Pipeline: GRPO algorithm implementation, group sampling, advantage computation, training loop.
- **Cheng Han Lin** – Experiments & Evaluation: benchmark design, baseline comparison, experiment execution, result analysis.

Detailed individual contributions will be summarized in the final report.

6.2 Timeline

Phase	Period	Tasks
Proposal	**Feb 17**	**Project proposal submission**
Phase 1: Setup	Feb 17 – Mar 2	Reproduce L2D baseline; set up codebase and environment (All members)
Phase 2: Core Dev	Mar 3 – Mar 23	Heng Jui: GNN encoder & JSSP environment; Kerstin: GRPO implementation; Cheng Han: evaluation framework & baselines
Progress Meeting	**Mar 24**	**Review proposal feedback, discuss challenges, present preliminary results, update goals**
Phase 3: Integration	Mar 25 – Apr 13	Integrate modules; initial end-to-end training and debugging (All members)
Phase 4: Experiments	Apr 14 – Apr 27	Run benchmarks and ablation studies; each member drafts their report sections
Phase 5: Report	Apr 28 – May 4	Consolidate report; review and finalize (All members)
Final Report	**May 5**	**Final report & code submission**

References

[1] Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P. S., & Xu, C. (2020). Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning. *arXiv preprint arXiv:2010.12367*.

[2] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., ... & Guo, D. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

[3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. *arXiv preprint arXiv:1707.06347*.

[4] Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How Powerful are Graph Neural Networks? *International Conference on Learning Representations (ICLR)*.

[5] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., ... & He, Y. (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.

[6] Ni, F., Zhang, J., & Deng, K. (2021). Dynamic Job-Shop Scheduling Problems Using Graph Neural Network and Deep Reinforcement Learning. *Proceedings of the IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 1–8.

[7] Park, J., Chun, J., Kim, S. H., Kim, Y., & Park, J. (2021). Learning to Schedule Job-Shop Problems: Representation and Policy Learning Using Graph Neural Network and Reinforcement Learning. *International Journal of Production Research*, 59(11), 3360–3377.

[8] Liu, H., Wang, Y., Fan, X., & Ye, Y. (2024). Multi-Agent Reinforcement Learning for Job Shop Scheduling in Dynamic Environments. *Sustainability*, 16(8), 3234.

[9] Zhang, C., Cao, Z., Song, W., Wu, Y., & Zhang, J. (2024). Deep Reinforcement Learning Guided Improvement Heuristic for Job Shop Scheduling. *arXiv preprint arXiv:2211.10936*.

[10] Fellek, G., Farid, A., Fujimura, S., Yoshie, O., & Gebreyesus, G. (2024). Gated-Attention Model with Reinforcement Learning for Solving Dynamic Job Shop Scheduling Problem. *Neurocomputing*, 579, 127392.

[11] Park, J., Bakhtiyar, S., & Park, J. (2021). ScheduleNet: Learn to solve multi-agent scheduling problems with reinforcement learning. *arXiv preprint arXiv:2106.03051*.

[12] Wang, L., Hu, X., & Wang, Y. (2024). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 20(2), 1234–1245.

[13] Yun, S., Jeong, M., Kim, R., Kang, J., & Kim, H. J. (2019). Graph Transformer Networks. *Advances in Neural Information Processing Systems*, 32.